

Finova Workspace — Complete Codebase Analysis Report

Project: Finova Workspace (my-v0-project) **Version:** 0.1.0 **Analysis Date:** 2026-06-23 **Prepared for:** Software Architecture & Technical Audit

Table of Contents

- 1. [Project Overview](#)
- 2. [Tech Stack Summary](#)
- 3. [Architecture Summary](#)
- 4. [Repository Structure](#)
- 5. [Frontend Analysis](#)
- 6. [Backend Analysis](#)
- 7. [Database Analysis](#)
- 8. [Authentication & Security](#)
- 9. [API Documentation](#)
- 10. [Data Flow Analysis](#)
- 11. [Feature Inventory](#)
- 12. [Dependency Analysis](#)
- 13. [Configuration Analysis](#)
- 14. [Deployment Analysis](#)
- 15. [Performance Review](#)
- 16. [Code Quality Review](#)
- 17. [Technical Debt & Risks](#)
- 18. [Security Assessment](#)
- 19. [Recommendations](#)
- 20. [Overall Codebase Health Score](#)

1. Project Overview

What Is This Project?

Finova Workspace is a full-featured, internal virtual office platform built for **Finova Solutions**. It serves as a centralized hub where employees, team leads, and executives collaborate through a single, unified dashboard.

Business Purpose

Replace scattered tools — separate chat apps, spreadsheet task trackers, email-chain announcements — with one integrated workspace.

Intended Users

Role	Access Level	Primary View
CEO	Full administrative	/admin/ceo — Company dashboard, user management, warnings, departments
C-Level	Management	/admin/c-level — Announcements, projects, team management

Team Lead	Team management	/admin/lead — Tasks, team performance, announcements
Employee	Self-service	/dashboard — Personal tasks, chat, calendar, files

Core Functionality

- **Role-based dashboards** — Different views and capabilities per role
- **Task Management** — Create, assign, track with priority, progress, due dates; Kanban board
- **Real-time Chat** — Direct messaging + group chat with polling
- **Calendar & Meetings** — Events/meetings with email invites, Google Calendar sync
- **Announcements** — Company-wide with read tracking
- **Project Management** — Status, progress, team assignments
- **Team Management** — Hierarchical structure with leads
- **Warning System** — Employee warnings with severity and resolution workflow
- **User Management** — CEO creates/edits/deletes users
- **Time Tracking** — Automatic for admin roles on login/logout
- **Notifications** — Bell icon with polling, dashboard popups

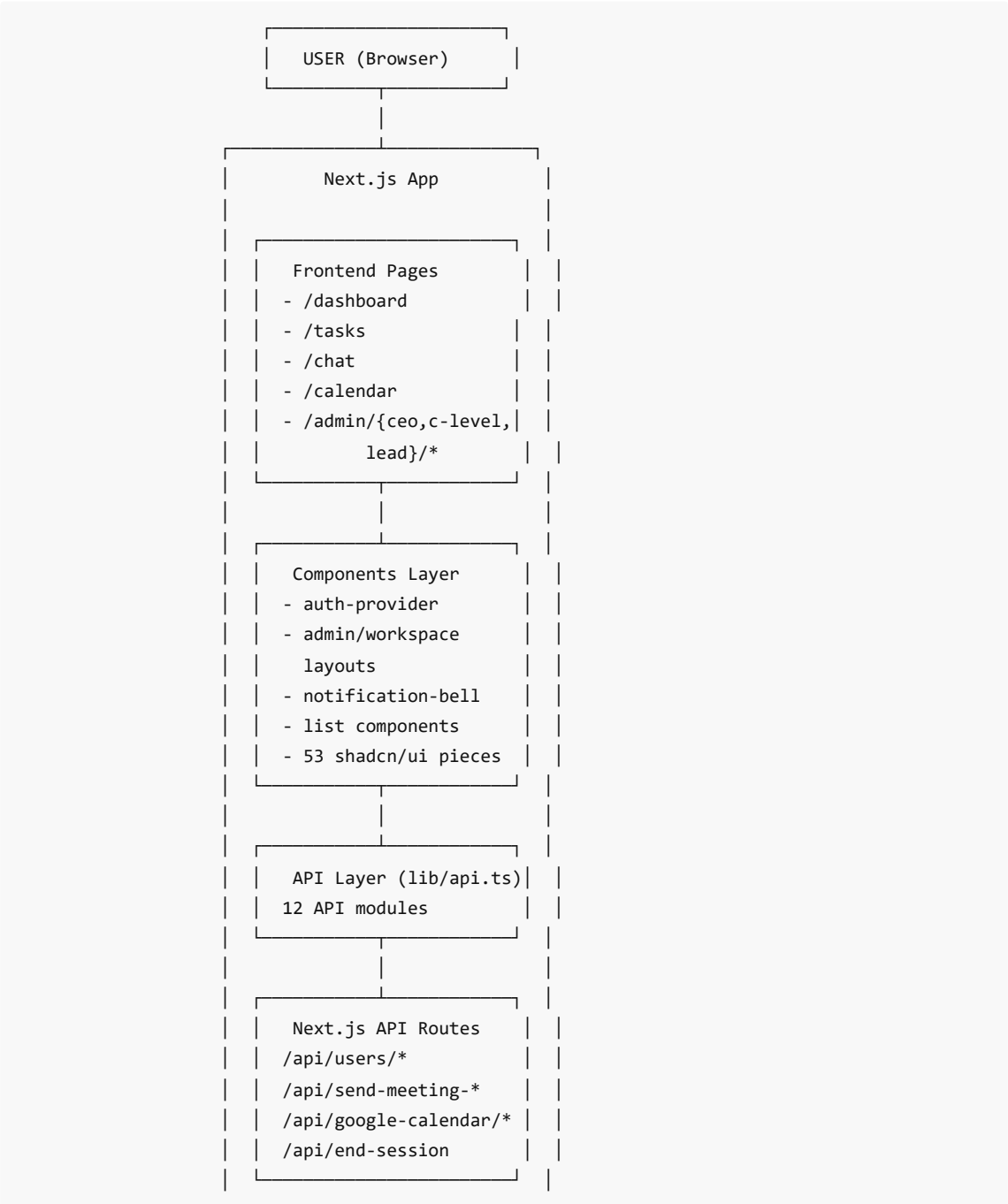
2. Tech Stack Summary

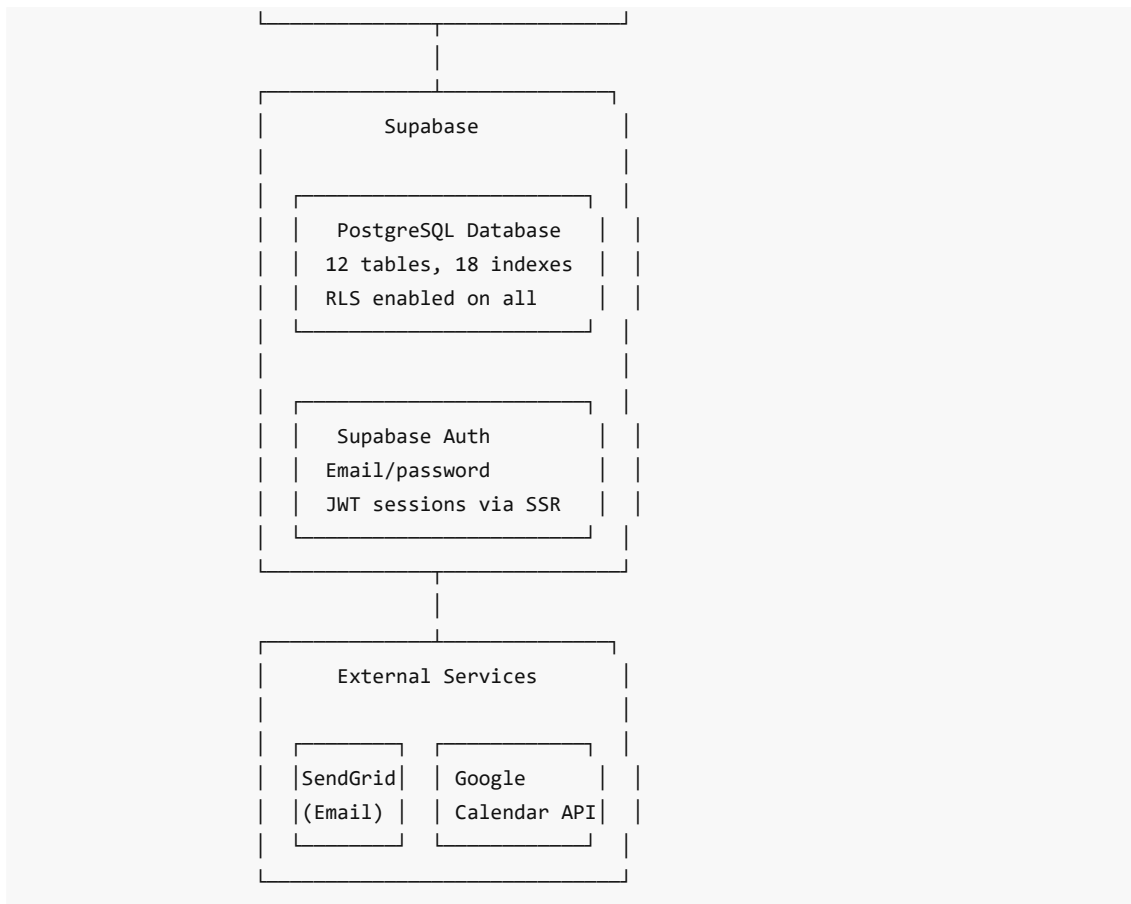
Layer	Technology	Version	Purpose
Framework	Next.js (App Router)	15.2.4	Full-stack React framework
Language	TypeScript	^5.0	Type safety
UI Library	React	^19	Component-based frontend
Styling	Tailwind CSS	^3.4.17	Utility-first CSS
UI Components	shadcn/ui (Radix)	—	53+ accessible UI primitives
Database	PostgreSQL (Supabase)	—	All data storage
Auth	Supabase Auth	—	Email/password + JWT sessions
Backend	Next.js API Routes	—	Serverless functions
Email	SendGrid (@sendgrid/mail)	^8.1.5	Meeting invitations
Calendar	Google APIs (googleapis)	^148.0.0	Google Calendar events
State	React Context	—	Auth state management
Animations	Framer Motion	^12.29.2	UI animations
Drag & Drop	dnd-kit	^6-10	Kanban board
Charts	Recharts	2.15.0	Analytics
Forms	React Hook Form + Zod	^7.54 / ^3.24	Form validation
Icons	Lucide React	^0.454.0	Icon set

Icons	Lucide React	^0.454.0	Icon set
Dates	date-fns	4.1.0	Date formatting
Theme	next-themes	^0.4.4	Dark/light mode
PM	npm / pnpm	—	Package management

3. Architecture Summary

Pattern: Client-Server (Monolithic with Next.js)





Key Architecture Decisions

1. **Monolithic Next.js** — Both frontend and backend live in the same project; API routes handle server-side operations
2. **Supabase as BaaS** — Handles database, authentication, and realtime; no separate backend server needed
3. **Role-based routing** — Route groups under `app/(workspace)/admin/{ceo,c-level,lead}/` with layouts that check user role
4. **Service role for admin ops** — User CRUD uses `SUPABASE_SERVICE_ROLE_KEY` for elevated privileges
5. **Polling over WebSockets** — Chat (10s) and notifications (30s) use polling rather than realtime subscriptions
6. **Not static export** — `next.config.mjs` explicitly removed `output: 'export'` to support API routes

4. Repository Structure

```
Finova-Workspace-main/
|
├─ app/                                # Next.js App Router
|   ├── layout.tsx                     # Root layout (ThemeProvider → AuthProvider → Toaster)
|   ├── page.tsx                       # Landing → redirects to /login
|   ├── globals.css                    # Global styles + Tailwind directives
|   └─ login/page.tsx                  # Login screen
|
|   └─ (workspace)/                    # Route group (authenticated)
|       ├── layout.tsx                 # Wraps AdminLayout with user role
|       └─ dashboard/page.tsx          # Employee dashboard
```

- ├─ announcements/{page,loading}.tsx
- ├─ calendar/page.tsx # Calendar + meeting creation
- ├─ chat/{page,loading}.tsx # Real-time chat
- ├─ files/{page,loading}.tsx # File manager (placeholder)
- ├─ projects/{page,loading}.tsx
- ├─ tasks/{page,loading}.tsx # Task list view
- ├─ tasks/kanban/ # Kanban board (dnd-kit)
- ├─ settings/page.tsx
- ├─ team/page.tsx
- ├─ exec/page.tsx
- └─ admin/
 - ├─ ceo/ # CEO dashboard + users/departments/warnings
 - ├─ c-level/ # C-Level dashboard
 - └─ lead/ # Lead dashboard
- └─ api/ # Next.js API routes
 - ├─ users/create/route.ts
 - ├─ users/update/route.ts
 - ├─ users/delete/route.ts
 - ├─ end-session/route.ts
 - ├─ send-meeting-invite/route.ts
 - └─ google-calendar/add-event/route.ts
- ├─ components/
 - ├─ ui/ # 53 shadcn/ui components
 - ├─ auth-provider.tsx # Auth context (login/logout/session)
 - ├─ admin-layout.tsx # Role-based sidebar navigation
 - ├─ workspace-layout.tsx # Employee sidebar
 - ├─ theme-provider.tsx # Dark/light mode
 - ├─ notification-bell.tsx # Notification dropdown
 - ├─ announcements-list.tsx
 - ├─ projects-list.tsx
 - ├─ teams-list.tsx
 - ├─ create-announcement.tsx
 - ├─ create-project-dialog.tsx
 - ├─ create-team-dialog.tsx
 - ├─ edit-team-dialog.tsx
 - └─ dashboard-card.tsx
- ├─ hooks/
 - ├─ use-mobile.tsx # Responsive breakpoints
 - └─ use-toast.ts # Toast notification hook
- ├─ lib/
 - ├─ supabase.ts # Supabase client initialization
 - ├─ types.ts # Database type definitions (540 lines)
 - ├─ api.ts # All API data access (2,236 lines)
 - ├─ firebase-types.ts # Legacy Firebase type aliases
 - └─ utils.ts # cn() utility (clsx + tailwind-merge)
- ├─ scripts/ # Build & deploy scripts
- └─ build-for-vps.ps1

```

|   ├── build-for-vps.sh
|   ├── deploy-check.js
|   ├── deploy-hostinger.sh
|   ├── seed-initial-users.ts
|   └── README.md
|
|   ├── public/                                # Static assets
|   |   ├── finova-icon.png
|   |   ├── finova-logo-full.png
|   |   ├── placeholder-*. {png,svg,jpg}
|   |   └── .htaccess
|   |
|   ├── styles/globals.css                    # Additional global styles
|   ├── logs/.gitkeep                        # Log directory
|   |
|   ├── SQL/                                # Database migrations (16 files)
|   |   ├── supabase-schema.sql              # Full schema
|   |   ├── reset-and-create-schema.sql
|   |   ├── add-rls-policies.sql
|   |   ├── add-select-policies.sql
|   |   ├── create-calendar-events-table.sql
|   |   ├── create-notifications-table.sql
|   |   ├── fix-* / seed-* / drop-*.sql
|   |   └── ...
|   |
|   ├── next.config.mjs                      # Next.js configuration
|   ├── tailwind.config.ts                   # Tailwind CSS configuration
|   ├── tsconfig.json                       # TypeScript configuration
|   ├── postcss.config.mjs                  # PostCSS configuration
|   ├── netlify.toml                        # Netlify deployment configuration
|   ├── ecosystem.config.js                  # PM2 process manager configuration
|   ├── package.json / package-lock.json / pnpm-lock.yaml
|   └── .gitignore

```

5. Frontend Analysis

Frontend Stack

Component	Technology
Framework	Next.js 15.2.4 (App Router)
Language	TypeScript ^5.0
Styling	Tailwind CSS 3.4 + CSS variables
UI Library	shadcn/ui (53 Radix-based components)
Icons	Lucide React
Animations	Framer Motion 12.29
Forms	React Hook Form + Zod

Drag & Drop	@dnd-kit/core/sortable
Charts	Recharts
State Management	React Context (AuthProvider) + local state
Theme	next-themes (dark/light mode)

Frontend Component Inventory

File	Purpose	Key Logic
app/layout.tsx	Root layout	Wraps app in ThemeProvider → AuthProvider; Toaster; sets Inter font
app/page.tsx	Landing page	Immediately redirects to /login
app/login/page.tsx	Login screen	Email/password form with validation; left p has animated background + InteractiveCat; right panel has login card; password visibility toggle
app/(workspace)/layout.tsx	Workspace wrapper	Reads user from useAuth(), renders <AdminLayout> if role={user.role}>
app/(workspace)/dashboard/page.tsx	Employee dashboard	Greeting (time-based), KPI cards (today's tasks assigned, messages, announcements), due today list, latest notifications; uses Framer Motion
app/(workspace)/admin/ceo/page.tsx	CEO dashboard	KPIs (active users, project/task completion, overdue), tabs (Departments/Projects/Users/Announcements), department performance progress bars, user task table
app/(workspace)/admin/c-level/page.tsx	C-Level dashboard	Stats cards, tabs for Announcements/Projects/Teams, CRUD dialog
app/(workspace)/admin/lead/page.tsx	Lead dashboard	KPIs, team member performance, task distribution, team overview, task creation, announcement creation
app/(workspace)/admin/ceo/users/page.tsx	User management	User table with search, create/edit/delete dialogs, role assignment, password validation
app/(workspace)/admin/ceo/departments/page.tsx	Department management	Department cards, create/edit/delete, member assignment, member counts
app/(workspace)/admin/ceo/warnings/page.tsx	Warning system	Issue/resolve/dismiss warnings, severity level active/resolved/all tabs
app/(workspace)/tasks/page.tsx	Task board	List view with tabs (All/To Do/In Progress/Review/Done), search filter, create

		dialog with multi-assignee, status transition buttons
<code>app/(workspace)/tasks/kanban/page.tsx</code>	Kanban board	Drag-and-drop columns (dnd-kit), task cards with assignee info
<code>app/(workspace)/chat/page.tsx</code>	Chat system	Direct messages + group chat, contact sidebar, message polling (10s), date grouping
<code>app/(workspace)/calendar/page.tsx</code>	Calendar	Month/week/day views, event/meeting creation with SendGrid + Google Calendar + dashboard notifications
<code>app/(workspace)/announcements/page.tsx</code>	Announcements	Read announcements, mark as read tracking
<code>components/auth-provider.tsx</code>	Auth context	User state, login/logout, session checking via Supabase, time tracking start/end on login/logout, role-based redirect, error message mapping
<code>components/admin-layout.tsx</code>	Admin sidebar	Collapsible sidebar, role-based nav items (C-Level gets Users/Departments/Warnings; C-Level gets Announcements/Projects/Teams; Lead gets Projects/Team), notification badge, user avatar dropdown
<code>components/workspace-layout.tsx</code>	Employee layout	Simplified sidebar (Dashboard/Files/Chat/Calendar/Tasks/Settings), search bar, notification bell
<code>components/notification-bell.tsx</code>	Notification dropdown	Polls every 30s, auto-opens on first unread, mark-as-read, sessionStorage for popup state

6. Backend Analysis

Backend Stack

Layer	Technology
Runtime	Node.js (Next.js server)
API Framework	Next.js API Routes (App Router)
Database Client	@supabase/supabase-js
Email Service	@sendgrid/mail (SendGrid)
Calendar Service	googleapis (Google Calendar API)
Auth Provider	Supabase Auth
Process Manager	PM2 (ecosystem.config.js)

API Module Architecture (lib/api.ts)

The entire data access layer is in a single **2,236-line file** containing 12 API modules:

Module	Functions	Key Tables	Lines
authAPI	login, checkSession, logout	users, Supabase Auth	~60
usersAPI	getAll, getUser, findUsersByEmails, updateUser, deleteUser	users	~100
projectsAPI	getAll, create, update, delete, createProject, updateProject, deleteProject	projects	~180
tasksAPI	getAll, getUserTasks, createTask, update, delete	tasks	~300
announcementsAPI	getAll, create, update, delete, markAsRead	announcements	~180
notificationsAPI	getUserNotifications, markAsRead, getUnreadCount, addNotification	notifications	~120
calendarEventsAPI	getAll, createEvent, updateEvent, deleteEvent	calendar_events	~200
chatAPI	sendDirectMessage, sendGroupMessage, getDirectMessageHistory, getGroupMessageHistory, getUserChatGroups, createChatGroup, markMessagesAsRead	chat_messages, chat_groups	~200
warningsAPI	getAll, create, updateStatus, getById	warnings	~120
timeTrackingAPI	startSession, endSession, cleanupStaleSessions, getAllSessions, getTimeAnalytics	time_tracking	~180
departmentsAPI	getAll, createDepartment, updateDepartment, deleteDepartment	departments	~80
teamsAPI	getAll, getDepartments, getTeamByLeadId, create, update, delete, getTeamWithMembers, addMember, removeMember, getAvailableUsers	teams, users	~250

Key Backend Logic Patterns

Snake_case to camelCase Mapping: Every data-fetching function maps database columns (created_at , lead_id) to frontend properties (createdAt , leadId). This is done repeatedly in every function rather than using a centralized mapper.

Role-Based Task Visibility:

```
if (userRole && ['CEO', 'C_LEVEL', 'LEAD'].includes(userRole)) {
  // No filter – see all tasks
} else {
  // Regular employees see assigned or viewable tasks
  query = query.or(`assignee_ids.cs.${userId}`,viewer_ids.cs.${userId}`)
}
```

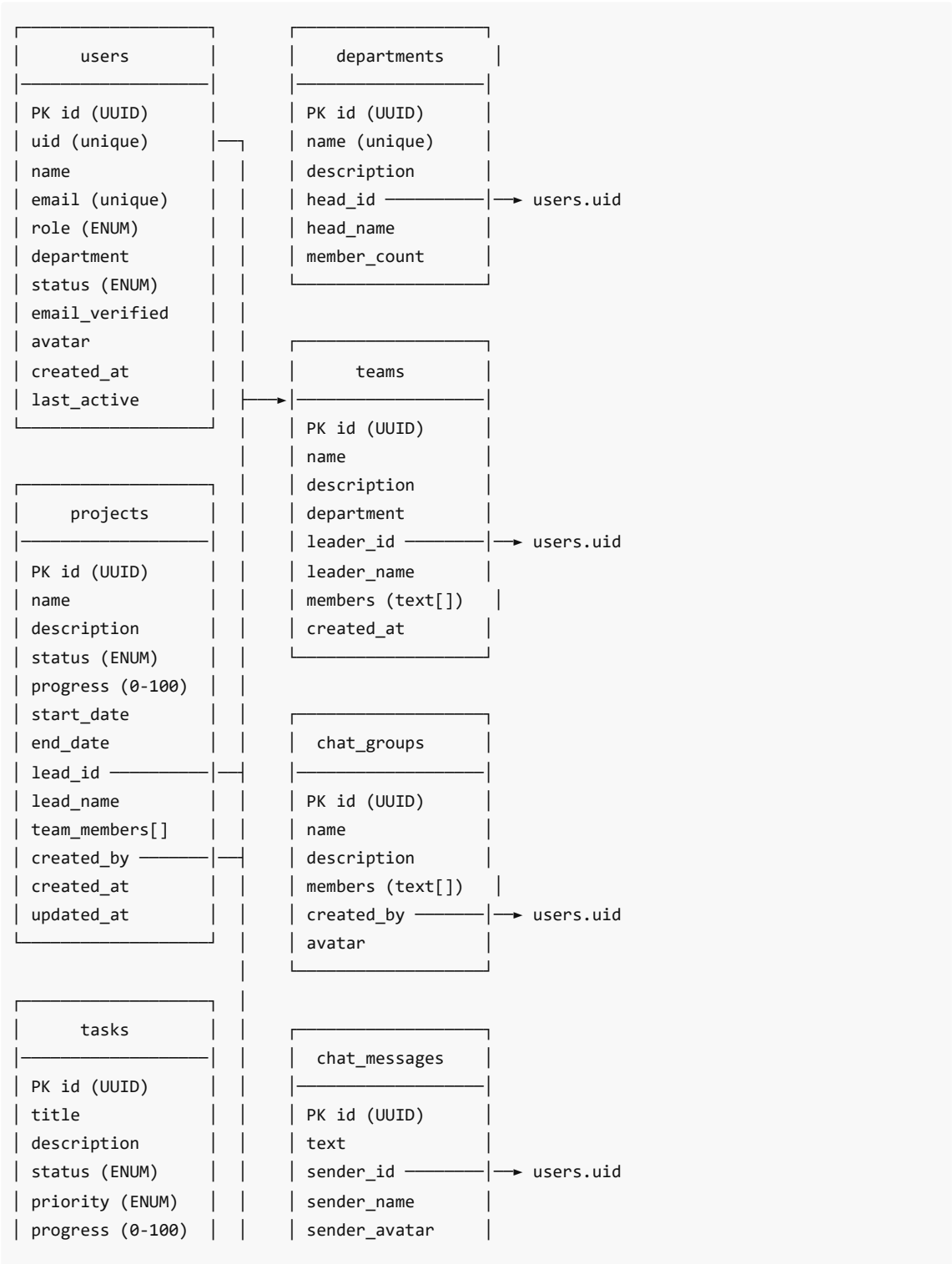
Backward Compatibility Aliases: The projectsAPI and tasksAPI maintain duplicate function signatures (create / createProject , update / updateTask) for backward compatibility with older code.

7. Database Analysis

Database: PostgreSQL (via Supabase)

12 Tables, 18 Indexes, RLS Enabled on All Tables

Entity Relationship Diagram



assignee_id ———	recipient_id ——— → users.uid
assignee_ids[]	group_id ——— → chat_groups.id
assignee_names[]	is_read
viewer_ids[]	created_at
viewer_names[]	
due_date	
project_id ———	
project_name	calendar_events
created_by ———	
created_by_name	PK id (UUID)
start_date	title
created_at	description
updated_at	start_time
	end_time
	location
	event_type (ENUM)
announcements	organizer_id ——— → users.uid
	organizer_name
PK id (UUID)	organizer_email
title	attendees[]
content	invited_member_ids
priority (ENUM)	(integration
author_id ———	flags: boolean)
author_name	created_at
read_by (text[])	
created_at	
updated_at	
	notifications
	PK id (UUID)
warnings	user_id ——— → users.uid
	type
PK id (UUID)	title
user_id ———	body
user_name	link_to
issuer_id ———	related_item_id
issuer_name	read (boolean)
title	created_at
description	
severity (ENUM)	
status (ENUM)	
department_id	time_tracking
department_name	
resolved_at	PK id (UUID)
resolved_by	user_id ——— → users.uid
resolved_reason	user_name
created_at	user_role
updated_at	start_time
	end_time
	duration (mins)
	is_active
	device (JSONB)

```

| | auto_ended |
|_|

```

Table Details

users — Core user profiles and role assignments

- `role` : EMPLOYEE | LEAD | C_LEVEL | CEO
- `status` : active | inactive | suspended
- Indexed on: `email` , `role` , `department`

projects — Company projects with tracking

- `status` : not_started | in_progress | completed | on_hold | cancelled
- `progress` : 0-100 integer
- Indexed on: `lead id` , `status`

tasks — Individual work items with multi-assignee support

- `status`: `todo` | `in_progress` | `in_review` | `completed`
- `priority`: `low` | `medium` | `high` | `urgent`
- Supports multiple assignees (`assignee_ids[]` , `assignee_names[]`) and viewers (`viewer_ids[]` , `viewer_names[]`)
- Indexed on: `assignee_id` , `project_id` , `status`

announcements — Company-wide communications

- `priority` : low | medium | high
- `read_by` : array of user IDs for read tracking

notifications — Per-user notification system

- `type` : string (meeting, task, warning, announcement)
- `read` : boolean flag
- Indexed on: `user_id`, `read`

chat_messages — Direct and group messages

- Polymorphic: either `recipient_id` (direct) or `group_id` (group)
- `is_read` : boolean
- Indexed on: `sender id` , `recipient id` , `group id`

chat groups — Group chat definitions

- members : array of user IDs

calendar_events — Calendar events and meetings

- `event_type`: `event` | `meeting`
- Integration flags: `send_calendar_invite`, `send_email_reminder`, `add_to_google_calendar`, `notify_on_dashboard`

warnings — Employee disciplinary tracking

- severity: low | medium | high | critical
- status: active | resolved | dismissed
- Indexed on: user id, status

time_tracking — Session-based work time tracking

- `is_active` : boolean for current session
- `auto_ended` : flag for stale session auto-cleanup
- `device` : JSONB metadata (userAgent, platform)
- Indexed on: `user_id` , `is_active`

departments — Organizational units

- `name` : unique
- `head_id` : references `users.uid`

teams — Sub-department teams

- `leader_id` : references `users.uid` ON DELETE SET NULL
- `members` : array of user IDs

8. Authentication & Security

Authentication Implementation

Component	Implementation
Auth Provider	Supabase Auth (email/password)
Session	JWT persisted via <code>@supabase/ssr</code> SSR cookies
Login Flow	<code>supabase.auth.signInWithPassword()</code> → fetch user data from <code>users</code> table
Logout Flow	<code>supabase.auth.signOut()</code> + end time tracking session
Session Check	<code>supabase.auth.getSession()</code> on mount + <code>onAuthStateChange</code> listener
Token Refresh	Auto-refresh via Supabase client config
Admin Operations	<code>SUPABASE_SERVICE_ROLE_KEY</code> used in API routes (server-only)

Security Assessment

Category	Status	Details
Password Hashing	✅	Handled by Supabase Auth (bcrypt)
Session Management	✅	JWT with SSR cookies
Email Verification	✅	<code>email_confirm</code> : true on admin user creation
Input Validation	⚠️ Partial	Login form validates email/password client-side; API routes check required fields
SQL Injection	✅	Supabase client uses parameterized queries
XSS Protection	✅	React's automatic escaping + Content Security Policy in <code>netlify.toml</code>

Row-Level Security	✓	All tables have RLS enabled
Rate Limiting	✗ Missing	No protection against brute force
Server-side Auth	⚠ Weak	API routes don't verify caller's role; any authenticated user could call user-management APIs if they knew the route
CORS	✗ Not configured	Not explicitly set
Security Headers	✓	Configured in netlify.toml (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)
Build-time Safety	✗ Disabled	ignoreBuildErrors: true and ignoreDuringBuilds: true suppress all errors

RLS Policy Analysis

Current Policies (Permissive):

- `users` : SELECT for all authenticated users → **anyone can see all user profiles, roles, emails**
- `projects` : SELECT for all authenticated users
- `tasks` : SELECT for all authenticated users
- `announcements` : SELECT for all authenticated users
- `notifications` : SELECT only where `user_id = auth.uid()` ← **properly scoped**
- `chat_messages` : SELECT where user is sender, recipient, or group member ← **properly scoped**

Risk: Sensitive data (user roles, project assignments, task assignments) is visible to all authenticated users. RLS policies should be tightened to match the role-based access enforced in the frontend.

9. API Documentation

Next.js API Routes

Method	Route	Purpose	Authentication	Request Body	Response
POST	<code>/api/users/create</code>	Create user in Auth + DB	Service Role Key	<code>{name, email, password, role, department}</code>	<code>{message, user}</code> (201)
PUT	<code>/api/users/update</code>	Update user profile	Service Role Key	<code>{userId, name?, role?, department?, status?}</code>	<code>{success, user}</code>
DELETE	<code>/api/users/delete</code>	Delete from Auth + DB	Service Role Key	<code>{userId}</code>	<code>{success, message}</code>
POST	<code>/api/end-session</code>	End time tracking session	User session	<code>{sessionId}</code>	<code>{success, result}</code>

POST	/api/send-meeting- invite	Send email with .ics invite	User session	{to[], cc[], subject, meetingDetails}	{success}
POST	/api/google- calendar/add-event	Add event to Google Calendar	User session	{title, description, startTime, endTime, location, attendees[]}	{success, eventId}

Client-Side API (lib/api.ts)

All client-side data access happens through 12 modules in `lib/api.ts`. Here are the key functions:

Authentication

- `authAPI.login(email, password)` → Login with Supabase Auth, fetch user profile
- `authAPI.checkSession()` → Verify current session, return user data
- `authAPI.logout()` → Sign out from Supabase

Users

- `usersAPI.getAll({role?, department?, search?})` → List users with filters
- `usersAPI.getUser(id)` → Single user by UID
- `usersAPI.findUsersByEmails(emails[])` → Find users by email array
- `usersAPI.updateUser(id, {name?, role?, department?, status?})` → Update via API route
- `usersAPI.deleteUser(id)` → Delete via API route

Tasks

- `tasksAPI.getAll({status?, assigneeId?, search?, userRole?, userId?})` → List tasks with role-based visibility
- `tasksAPI.getUserTasks(userId, userRole?)` → Tasks visible to a specific user
- `tasksAPI.createTask(data)` → Create with multi-assignee/viewer support
- `tasksAPI.update(id, data)` → Update task fields
- `tasksAPI.delete(id)` → Delete task

Calendar Events

- `calendarEventsAPI.getAll(userId)` → Events where user is organizer or invitee
- `calendarEventsAPI.createEvent(event)` → Create event with all integration flags
- `calendarEventsAPI.updateEvent(id, updates)` → Update event
- `calendarEventsAPI.deleteEvent(id)` → Delete event

Chat

- `chatAPI.sendDirectMessage(senderId, recipientId, text, senderName?, senderAvatar?)` → Send DM
 - `chatAPI.sendGroupMessage(senderId, groupId, text, senderName?, senderAvatar?)` → Send group message
 - `chatAPI.getDirectMessageHistory(userId1, userId2, limit?)` → DM history
 - `chatAPI.getGroupMessageHistory(groupId, limit?)` → Group message history
 - `chatAPI.getUserChatGroups(userId)` → User's groups
 - `chatAPI.createChatGroup({name, description, members, createdBy})` → Create group
-

10. Data Flow Analysis

User Login Flow

1. User navigates to /login
2. Login page renders left panel (branding + InteractiveCat animation) + right panel (login form)
3. User enters email and password
4. Client-side validation:
 - Email format (regex: /^[^\s@]+@[^\s@]+\.[^\s@]+\$/)
 - Password minimum 6 characters
5. User clicks "Login" → form submit
6. authProvider.login(email, password) is called
7. authAPI.login() calls supabase.auth.signInWithPassword({email, password})
8. Supabase validates credentials, returns JWT session
9. If auth fails → error message mapped to user-friendly text ("Incorrect email or password", etc.)
10. If auth succeeds → fetch user data from users table WHERE uid = authData.user.id
11. If user not found in users table → return needsOnboarding: true
12. If user found → set user state in AuthContext
13. If admin role (CEO/C_LEVEL/LEAD) → start time tracking session
14. Toast shows "Welcome back, [name]!"
15. useEffect redirects based on role:
 - CEO → /admin/ceo
 - C_LEVEL → /admin/c-level
 - LEAD → /admin/lead
 - EMPLOYEE → /dashboard
16. AdminLayout/Workspacelayout renders with role-appropriate sidebar

Task Creation Flow (Admin)

1. Admin navigates to /tasks
2. TasksPage fetches tasks via tasksAPI.getUserTasks(user.id, user.role)
3. Admin clicks "Create Task" → dialog opens
4. Admin fills form: title, description, assignees, viewers, priority, due date
5. Admin clicks "Assign Task"
6. tasksAPI.createTask() validates required fields (assigneeIds, assigneeNames, createdBy, createdByName)
7. Supabase insert into tasks table:
 - assignee_ids[], assignee_names[], viewer_ids[], viewer_names[]
 - Legacy columns for backward compat (assignee_id, assignee_name)
8. On success → toast "Task assigned to [names]"
9. Task list refreshed via fetchTasks(false) – no loading spinner

Chat Message Flow

1. User navigates to /chat
2. ChatPage fetches all users (excluding current), stored in state
3. User selects contact from sidebar
4. chatAPI.getDirectMessageHistory(user.id, contact.uid) fetches messages

- 5. Messages displayed with date grouping, sender styling, timestamps
- 6. Polling interval starts (10s) for new messages
- 7. User types message, presses Enter
- 8. chatAPI.sendMessage(senderId, recipientId, text, senderName)
- 9. Supabase insert into chat_messages table
- 10. Input cleared immediately for UX
- 11. Immediate re-fetch to show sent message
- 12. On selecting a conversation → unread messages marked as read
- 13. localStorage saves last selected chat for persistence

Calendar Event Creation Flow (Meeting)

- 1. User navigates to /calendar
- 2. CalendarPage fetches events via calendarEventsAPI.getAll(user.id)
- 3. User clicks "New Event" → dialog opens
- 4. User selects "meeting" type
- 5. User fills: title, description, date, time, location, attendees (email), CC employees
- 6. User selects team members to invite (checkbox list)
- 7. User toggles integration options:
 - Send calendar invitation (SendGrid email with .ics)
 - Send email reminder
 - Add to Google Calendar
 - Notify on dashboard
- 8. User clicks "Create Meeting"
- 9. calendarEventsAPI.createEvent() inserts into calendar_events table
- 10. If sendCalendarInvite → POST /api/send-meeting-invite → SendGrid sends email
- 11. If addToGoogleCalendar → POST /api/google-calendar/add-event → Google Calendar API
- 12. If notifyOnDashboard → notificationsAPI.addNotification() for each invited member
- 13. Toast shows "Meeting has been created"

11. Feature Inventory

Feature	Purpose	Frontend Files	Backend Files	DB
Authentication	Login/logout/session	auth-provider.tsx, login/page.tsx	api.ts (authAPI)	users, Supab
User Management	CRUD users, roles	admin/ceo/users/page.tsx	api/users/*/route.ts, api.ts (usersAPI)	users
Task Management	Create/assign/track tasks	tasks/page.tsx, tasks/kanban/*	api.ts (tasksAPI)	tasks
Kanban Board	Drag-and-drop task view	tasks/kanban/{page,kanban-column,task-card,sortable-task-card}.tsx	api.ts (tasksAPI.update)	tasks
Chat	Direct & group messaging	chat/page.tsx	api.ts (chatAPI)	chat_m chat_gi
Calendar	Events & meetings	calendar/page.tsx	api.ts (calendarEventsAPI),	calendi

			api/google-calendar/*, api/send-meeting-invite/*	
Announcements	Company communications	announcements/page.tsx, announcements-list.tsx, create-announcement.tsx	api.ts (announcementsAPI)	announc
Projects	Project tracking	projects/page.tsx, projects-list.tsx, create-project-dialog.tsx	api.ts (projectsAPI)	project
Teams	Team structure management	teams-list.tsx, create-team-dialog.tsx, edit-team-dialog.tsx	api.ts (teamsAPI)	teams
Departments	Org structure	admin/ceo/departments/page.tsx	api.ts (departmentsAPI)	departm
Warning System	Employee discipline	admin/ceo/warnings/page.tsx	api.ts (warningsAPI)	warning notific
Notifications	Bell alerts	notification-bell.tsx	api.ts (notificationsAPI)	notific
Time Tracking	Session-based tracking	auth-provider.tsx, api/end-session/route.ts	api.ts (timeTrackingAPI)	time_tr
Role-Based Dashboards	Per-role views	admin-layout.tsx, workspace-layout.tsx, admin/{ceo,c-level,lead}/page.tsx	—	—
Dark/Light Mode	Theme switching	theme-provider.tsx, tailwind.config.ts	—	—

12. Dependency Analysis

Production Dependencies

Package	Version	Size (approx.)	Purpose	Critical?
next	15.2.4	Large	Framework	✅ Yes
react / react-dom	^19	Large	UI Engine	✅ Yes
@supabase/supabase-js	^2.93.2	Medium	Database Client	✅ Yes
@supabase/ssr	^0.8.0	Small	Auth Sessions	✅ Yes
tailwindcss	^3.4.17	Medium	Styling	✅ Yes
@sendgrid/mail	^8.1.5	Medium	Email Service	⚠️ Optional

googleapis	^148.0.0	Large	Google Calendar	⚠️ Optional
@dnd-kit/*	^6-10	Small	Drag & Drop	⚠️ Kanban only
framer-motion	^12.29.2	Medium	Animations	⚠️ Non-critical
recharts	2.15.0	Medium	Charts	⚠️ Non-critical
lucide-react	^0.454.0	Small	Icons	⚠️ Non-critical
@radix-ui/* (26 pkg)	various	Medium	UI Primitives	⚠️ Non-critical
react-hook-form	^7.54.1	Small	Forms	⚠️ Underutilized
zod	^3.24.1	Small	Validation	⚠️ Underutilized
cmdk	1.0.4	Small	Command Palette	❌ Unused
input-otp	1.4.1	Small	OTP Input	❌ Unused
@opentelemetry/api	^1.9.0	Small	Observability	❌ Unused
@hookform/resolvers	^3.9.1	Small	Form resolvers	❌ Unused
sonner	^1.7.1	Small	Toasts	❌ Unused (uses custom toast)
vaul	^0.9.6	Small	Drawer	❌ Unused
embla-carousel-react	8.5.1	Small	Carousel	❌ Unused
resend	—	Small	Email	❌ Not in deps (SendGrid used)

Dev Dependencies

Package	Version	Purpose
typescript	^5	Type checking
@types/node, @types/react, @types/react-dom	^22/^19	Type definitions
postcss	^8	CSS processing
tsx	^4.19.2	TypeScript execution for scripts
@netlify/plugin-nextjs	^5.7.4	Netlify deployment
dotenv	^16.4.7	Environment variable loading

Issues Found

- 10+ unused dependencies** — `cmdk`, `input-otp`, `@opentelemetry/api`, `@hookform/resolvers`, `sonner`, `vaul`, `embla-carousel-react` are installed but never imported in any source file
- Duplicate toast systems** — Both `sonner` (package) and custom `use-toast` / `toaster` (manual implementation) exist; only the manual one is used

3. **Both npm and pnpm lock files** — `package-lock.json` and `pnpm-lock.yaml` both present, indicating mixed usage
4. **Large googleapis package** — Used only for a single API route (Google Calendar) that requires OAuth configuration to work

13. Configuration Analysis

Environment Variables Required

Variable	Purpose	Required
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL	✓ Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous API key	✓ Yes
SUPABASE_SERVICE_ROLE_KEY	Supabase service role key (admin operations)	✓ Yes (for API routes)
SENDGRID_API_KEY	SendGrid API key for email sending	⚠ Optional (graceful fallback)
GOOGLE_CLIENT_ID	Google OAuth client ID	⚠ Optional
GOOGLE_CLIENT_SECRET	Google OAuth client secret	⚠ Optional
GOOGLE_REDIRECT_URI	Google OAuth redirect URI	⚠ Optional
GOOGLE_REFRESH_TOKEN	Google OAuth refresh token	⚠ Optional

Build Configuration

next.config.mjs:

- `eslint.ignoreDuringBuilds: true` — All ESLint errors suppressed
- `typescript.ignoreBuildErrors: true` — All TypeScript errors suppressed
- `images.remotePatterns: [{ hostname: '**' }]` — Allows images from any domain
- Static export explicitly disabled to support API routes

tsconfig.json:

- Target: ES6
- Strict mode enabled
- Path alias: `@/*` → `./*`
- JSX: `preserve` (Next.js handles compilation)

tailwind.config.ts:

- Dark mode: `class` strategy
- Custom colors via CSS variables (background, foreground, card, popover, primary, secondary, muted, accent, destructive, border, input, ring, chart, sidebar)
- Custom animations: accordion-down/up

netlify.toml:

- Build command: `npm run build`
- Publish directory: `.next`

- Plugin: `@netlify/plugin-nextjs`
- Node bundler: `esbuild`
- Security headers configured
- Catch-all redirect to `index.html`

ecosystem.config.js (PM2):

- App name: `finova-workspace`
- Runs: `npm start` on port 3000
- 1 instance, 1GB max memory
- Auto-restart enabled
- Log files in `./logs/`

14. Deployment Analysis

Deployment Targets

Primary: Netlify

- Configured via `netlify.toml`
- Uses `@netlify/plugin-nextjs` for optimal support
- Build: `npm run build`
- Security headers (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)
- API routes automatically become Netlify Functions

Secondary: VPS (Hostinger)

- Configured via `ecosystem.config.js` (PM2)
- Scripts: `scripts/build-for-vps.ps1` (Windows) and `scripts/build-for-vps.sh` (Linux)
- Deployment: `scripts/deploy-hostinger.sh`
- Runs on port 3000 with PM2 process manager
- Health check: `scripts/deploy-check.js`

CI/CD

- No CI/CD configuration found (no GitHub Actions, no CircleCI, no Jenkins files)
- Deployment appears manual via scripts

Deployment Workflow (VPS)

1. `npm run build` → Build Next.js app
2. PM2 starts app → `ecosystem.config.js`
 - └─ name: `finova-workspace`
 - └─ port: `3000`
 - └─ instances: `1`
 - └─ max_memory: `1G`
 - └─ auto-restart: `true`
3. Logs → `./logs/`

15. Performance Review

Rendering Performance

Aspect	Assessment	Details
Client-side rendering	⚠️ All pages are "use client"	No React Server Components used; all pages render on the client
Bundle size	⚠️ Large	googleapis (large package), framer-motion, recharts, and 53+ UI components imported
Animation overhead	⚠️ Heavy	Framer Motion used on dashboard, lists, modals; animate-pulse on background blur elements
Image optimization	✅ Good	Next.js Image component configured with remotePatterns: ['**']

Network Performance

Aspect	Assessment	Details
API calls	⚠️ No batching	Many pages fire 3-4 parallel Supabase queries (e.g., CEO dashboard: Promise.all([users, projects, tasks, announcements]))
Chat polling	⚠️ Inefficient	Polls every 10s for new messages; re-fetches entire history
Notification polling	⚠️ Inefficient	Polls every 30s; queries both notifications list and unread count
Data overhead	⚠️ No selective fetching	getAll() functions always SELECT * with no column selection

Database Efficiency

Aspect	Assessment	Details
Indexes	✅ Good	18 indexes on foreign keys and filtered columns
Query patterns	⚠️ Some inefficient	array.contains() operations on text arrays (assignee_ids, viewer_ids)
RLS overhead	⚠️ Present	RLS policies add query overhead on every request

Bottlenecks Identified

- No pagination** — All list queries (getAll) fetch all records without limit
- No caching** — Every page load triggers fresh Supabase queries; no Redis or in-memory cache
- Chat polling** — 10-second polling with full history re-fetch is inefficient for scale
- Large bundle** — Unused dependencies bloat initial JS payload
- No React Server Components** — All pages are client components, preventing server-side rendering benefits

16. Code Quality Review

Category	Score (0-10)	Notes
----------	--------------	-------

Structure & Organization	7/10	Well-organized folder structure (app/components/lib/hooks); some duplication across admin pages
Readability	7/10	Generally readable; some files are very long (api.ts: 2,236 lines; calendar/page.tsx: 1,247 lines)
Maintainability	5/10	High duplication (snake_case→camelCase mapping repeated in every function); mixed naming conventions
Naming Conventions	6/10	Inconsistent: some functions use createTask, others create; database uses snake_case, frontend uses camelCase
Reusability	5/10	UI components are reusable (shadcn/ui); business logic is tightly coupled to pages
Error Handling	6/10	Good try/catch patterns but with redundant console.error logging
TypeScript Usage	7/10	Well-typed database types (540 lines of types); some any types in page components
Testing	0/10	No test files found (no Jest, no Vitest, no Playwright, no testing directory)
Documentation	4/10	Some inline comments; no README at project root (only in /scripts); no API documentation
Code Duplication	4/10	High: snake_case→camelCase mapping repeated ~30 times; admin page layouts share significant code

Key Code Quality Issues

1. **lib/api.ts is 2,236 lines** — A single file containing 12 API modules should be split into separate files
2. **Snake_case→camelCase mapping duplicated everywhere** — Every data-fetching function manually maps response fields instead of using a centralized transformation
3. **Duplicate function aliases** — `projectsAPI.create()` and `projectsAPI.createProject()` do the same thing
4. **Dead code** — Firebase timestamp handling in `notification-bell` (`timestamp.toDate ? timestamp.toDate() : ...`) references a legacy Firebase pattern
5. **Build errors suppressed** — `ignoreBuildErrors: true` and `ignoreDuringBuilds: true` hide real issues
6. **useEffect dependency warnings suppressed** — Multiple `// eslint-disable-next-line` comments used instead of fixing dependencies

17. Technical Debt & Risks

Critical

Issue	Location	Impact
No server-side authorization	All API routes	Any authenticated user could call admin APIs

RLS policies too permissive	supabase-schema.sql	SELECT=true for users/projects/tasks/announcements exposes sensitive data
Build errors suppressed	next.config.mjs	Hidden TypeScript and ESLint errors will surface in production
No tests anywhere	Entire project	Zero test coverage; regressions guaranteed on changes

High

Issue	Location	Impact
lib/api.ts monolithic file	2,236-line single file	Hard to maintain, test, or reason about
No pagination on any list query	All getAll() functions	Will break with thousands of records
Chat polling (10s)	chat/page.tsx	Does not scale; should use Supabase Realtime or WebSockets
No input sanitization	All user-facing inputs	XSS risk in announcements, tasks, chat messages
Mixed npm/pnpm lock files	Root directory	Conflicting dependency resolution

Medium

Issue	Location	Impact
10+ unused dependencies	package.json	Bloating bundle size
No React Server Components	All pages are "use client"	No SSR/SSG benefits; larger client bundles
Duplicate toast systems	components/ui/toast.tsx VS sonner	Confusion; one system unused
Duplicated API functions	projectsAPI.create/createProject	Maintenance burden
Firebase legacy patterns	Notification-bell, calendar-page	Dead code from previous Firebase migration

Low

Issue	Location	Impact
Inconsistent naming	Mixed camelCase/snake_case	Readability issue
Hardcoded URLs	HOSTINGER_DEPLOYMENT_COMPLETE.md	Deployment brittleness
.env.example missing	Root directory	New developers can't easily set up

18. Security Assessment

Category	Rating	Details
Authentication	✅ Strong	Supabase Auth with JWT, SSR sessions, auto-refresh
Authorization	❌ Weak	No server-side role enforcement; frontend-only UI hiding
Data Protection	⚠️ Partial	RLS enabled but policies are permissive
Input Validation	⚠️ Partial	Client-side only; no server-side sanitization
API Security	⚠️ Weak	No rate limiting, no CORS, no request validation middleware
Dependency Security	⚠️ Monitor	80+ dependencies; no automated vulnerability scanning
Secret Management	⚠️ Partial	Service role key used in API routes (acceptable); no mention of .env setup docs
Security Headers	✅ Configured	In netlify.toml (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)

19. Recommendations

Immediate (Must Fix)

1. **Enable TypeScript and ESLint checks in build** — Remove `ignoreBuildErrors` and `ignoreDuringBuilds` from `next.config.mjs` ; fix all surfaced errors
2. **Add server-side authorization to API routes** — Verify caller's role matches the operation (e.g., only CEO can call `POST /api/users/create`)
3. **Tighten RLS policies** — Match database-level security to the role-based UI hierarchy
4. **Add rate limiting** — Protect login endpoint and API routes from brute force

Short-Term (Should Fix)

5. **Split `lib/api.ts`** — Separate into `lib/api/auth.ts` , `api/tasks.ts` , `api/users.ts` , etc.
6. **Remove unused dependencies** — `cmdk` , `input-otp` , `@opentelemetry/api` , `@hookform/resolvers` , `sonner` , `vaul` , `embla-carousel-react`
7. **Add pagination** — All `getAll()` functions should accept `limit` and `offset`
8. **Replace chat polling with Supabase Realtime** — More efficient, real-time messaging
9. **Create centralized snake_case→camelCase mapper** — Stop duplicating mapping logic
10. **Remove Firebase legacy code** — Clean up `toDate()` fallbacks and Firebase references

Medium-Term (Should Consider)

11. **Adopt testing** — Add Vitest for unit tests, Playwright for E2E
12. **Add CI/CD pipeline** — GitHub Actions for linting, typecheck, tests on every PR
13. **Create `.env.example`** — Document all required environment variables
14. **Add React Server Components** — Convert static content to RSCs for better performance
15. **Create comprehensive README** — Project setup, architecture docs, deployment guide

20. Overall Codebase Health Score

Score: 58 / 100

Score Breakdown

Category	Weight	Score	Weighted
Architecture & Structure	15%	7/10	10.5
Code Quality	15%	5/10	7.5
Testing	10%	0/10	0
Security	15%	5/10	7.5
Performance	10%	5/10	5
Documentation	5%	4/10	2
Maintainability	10%	5/10	5
Dependency Management	5%	6/10	3
Deployment Readiness	10%	7/10	7
Error Handling	5%	6/10	3
Total	100%		58/100

Score Justification

Strengths (+):

- Well-organized Next.js App Router structure with clear route groups
- Comprehensive feature set covering 15+ features
- Proper database schema with indexes and RLS
- Good use of TypeScript for type safety
- Multiple deployment targets (Netlify + VPS)
- Robust authentication via Supabase

Weaknesses (-):

- Zero test coverage — no tests exist anywhere in the project
- Build-time safety disabled — TypeScript and ESLint errors suppressed
- Security gaps — no server-side authorization, permissive RLS, no rate limiting
- Massive single-file API layer (2,236 lines)
- 10+ unused dependencies bloating the bundle
- Chat and notification systems use inefficient polling
- No pagination on any data-fetching operation
- Firebase legacy code remains post-migration
- Documentation severely lacking (no project README, no API docs)

Verdict: The project has solid bones — good architecture, comprehensive features, modern stack — but is weighed down by technical debt from a likely rapid development cycle. The lack of tests and suppressed build errors are the most critical issues that will compound over time.

End of Report